

TREKKING MANAGEMENT APPLICATION

(Modern Application Development-1)

Name: **Sheetal Bajaj** Roll Number: **25f1001114**

1. Introduction

Adventure tourism has gained significant popularity over the past decade. Managing trekking events manually often creates several challenges, including duplicate bookings, payment tracking difficulties, poor communication, and inefficient participant management.

The Trekking Management Application addresses these issues by providing a centralized digital solution that automates the complete trekking management process. The application provides secure authentication, online trek booking, gallery management, payment simulation, and attendance management while ensuring proper role-based authorization.

2. Objectives

The major objectives of the project are:

- Develop a centralized trekking management system.
- Eliminate manual booking and management processes.
- Implement secure user authentication using Flask-Login.
- Restrict system functionality using role-based authorization.
- Allow administrators to create, edit, and manage trekking events.
- Allow trekkers to browse and book available treks.
- Enable online payment simulation using Google Pay, PhonePe, and UPI.
- Build a fully responsive interface.
- Prevent duplicate bookings through server-side validation.

3. Technology Stack

Component	Technology
Programming Language	Python 3
Backend Framework	Flask
ORM	SQLAlchemy
Database	SQLite
Authentication	Flask-Login
Frontend	HTML5, CSS3, Bootstrap 5
Client-side	JavaScript

4. System Modules

- **AdministratorModule:** Full control over treks, users, staff assignment, and booking monitoring.

- Admin credentials for testing are mentioned below:

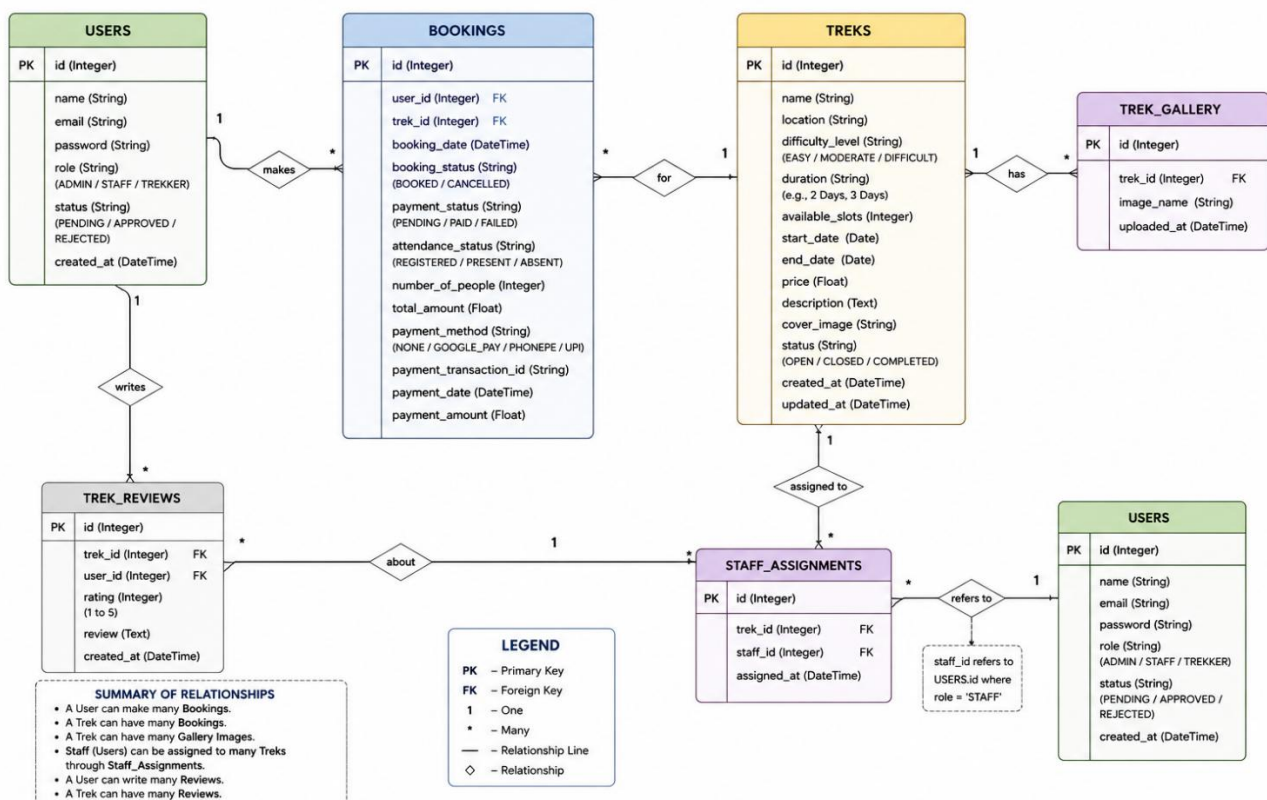
```
email="admin@trailsync.com",
password="admin123"
```

- **Trekker Module:** Browse treks, book adventures, process payments, and view history.
- **Staff Module:** View assigned treks and manage participant attendance.
- **AuthenticationModule:** Secure login, logout, and role-based session management.

5. Database Design (ER Diagram)

The application uses SQLite as the backend database, managed via SQLAlchemy .

ER DIAGRAM – TREKKING MANAGEMENT APPLICATION



6. API Endpoints

Method	Endpoint	Description
GET	/login	Login page
POST	/register	User registration
GET	/trekker/dashboard	Dashboard
GET	/trekker/book/<trek_id>	Book trek
GET	/trekker/my-bookings	Booking history

GET	/trekker/payment/<booking_id>	Payment page
GET	/trekker/payment/success/<booking_id>	Payment success
GET	/admin/create-trek	Create trek

7. Security Features

- **Authentication:** Flask-Login for secure session management.
- **Authorization:** Role-based route protection.
- **Password Security:** Robust password hashing.
- **CSRF Protection:** Implemented via Flask-WTF.

8. Payment Module

The application implements a payment simulation suitable for academic demonstration. This process mimics real-world payment gateways by transitioning booking states from 'PENDING' to 'PAID' via generated transaction IDs.

9. Responsive User Interface

Utilizing Bootstrap 5 and custom CSS media queries, the application is fully responsive across desktops, tablets, and smartphones.

10. Testing

- **Authentication:** Tested registration, login, and session persistence.
- **Trek Management:** Verified CRUD operations and image upload.
- **Booking:** Tested duplicate booking validation.
- **Payment:** Simulated payment lifecycle.

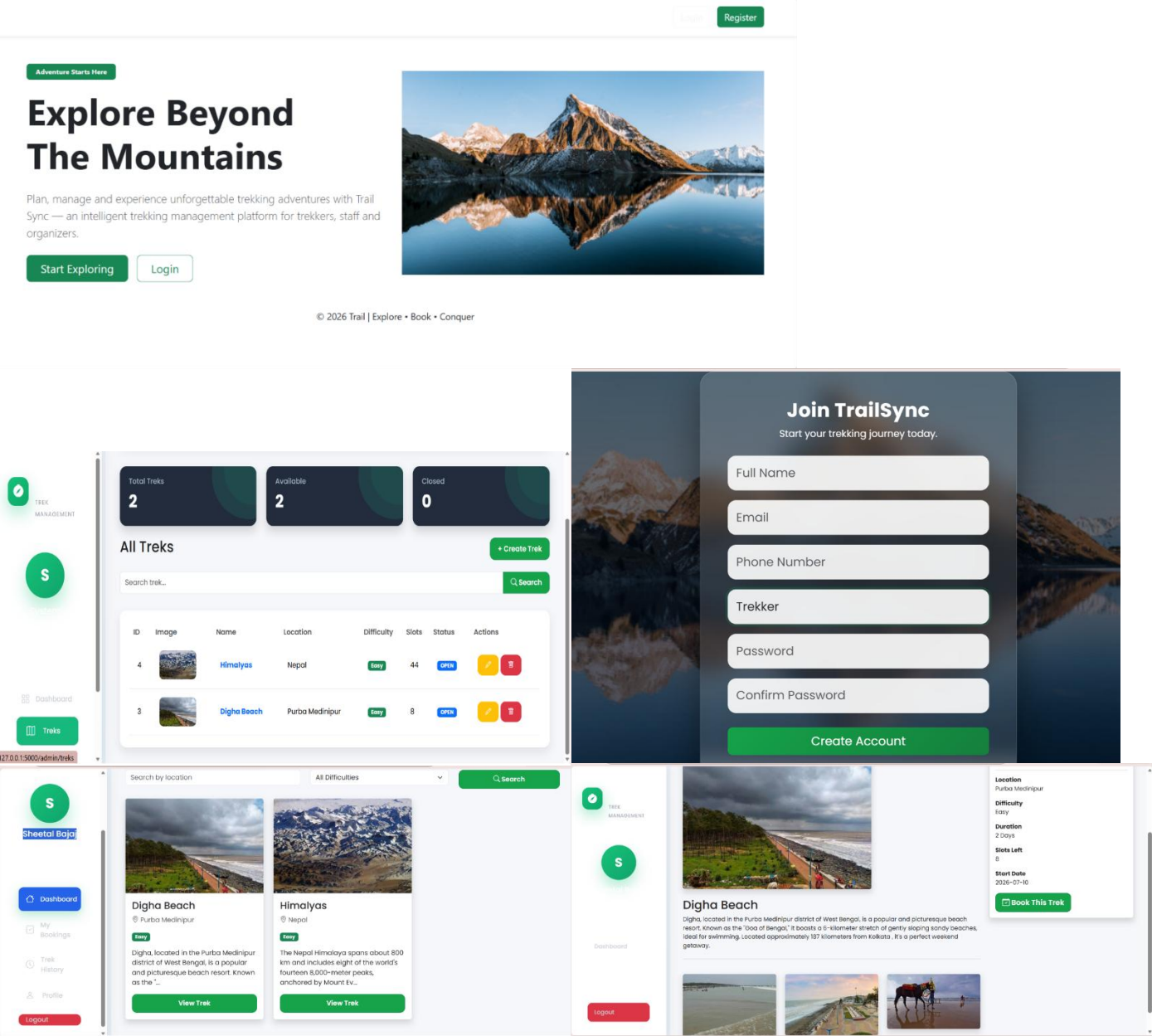
11. Challenges Faced

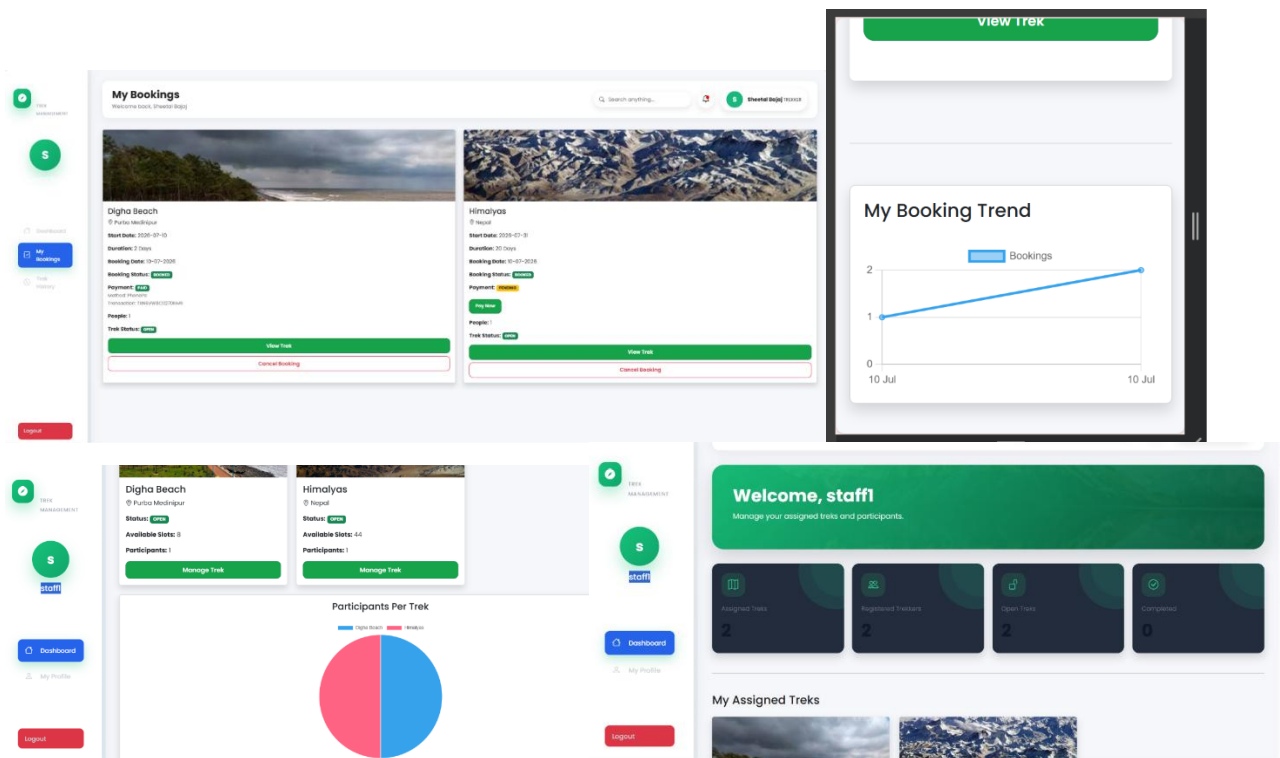
Key challenges included Flask-Login integration, managing complex SQLAlchemy relationships, and ensuring the sidebar remained responsive.

12. Future Enhancements

Planned improvements include real-world Razorpay integration, SMS/Email notifications, and AI-based recommendations.

13. Project Screenshots





14. AI Declaration:

This project was developed primarily through my own design, implementation, testing, and debugging efforts. AI tools (primarily ChatGPT) were used as a development assistant for concept clarification, debugging support, responsive design (media queries), security-related guidance, database implementation, documentation, and implementation suggestions.

Estimated AI Assistance: 25%

All AI-generated suggestions were carefully reviewed, tested, and modified where necessary before being incorporated into the project. The overall architecture, feature implementation, integration, testing, and final decisions were completed by me, and I take full responsibility for the final codebase and documentation.

15. Conclusion

The Trekking Management Application successfully demonstrates a secure, full-stack web application. It fulfills all intended objectives, providing a robust foundation for future scaling and professional deployment.

16. Presentation Video Link:

https://drive.google.com/file/d/1PCyfuHYTW20w5VirDvVzUWcZlr_0k5rB/view?usp=sharing

THANK YOU!